



Introduction To Betpro Flutter Full app

Betpro Flutter Full app is as the name says is dominating and powerful set of Flutter apps containing complete app templates, complete API integration with Betpro, ready-to-use widgets, and code snippets i.e. readymade code integration and stunning screens covering many different use cases. The app comes with the latest material design. Betpro Flutter Full app UI templates are developed with the highest quality, ease of reusing widgets, fast, and completely user-friendly interface. You can easily integrate these into your betting app. Betpro Flutter Full app makes your job easy to achieve the modern look and feel of the mobile application. It saves your hustle and time from developing a betting for your betting website made with betpro.

Prerequisite:

1 Android Studio: 4.1.2

2 Flutter: 3.0.0

Tools & Setup

Prerequisite

- Flutter & Dart [SDK](#)
- Any IDE Android Studio (Recommended), [Visual Studio Code](#) or [IntelliJ IDEA](#)
- To edit this project you must have Flutter and Dart installed and configured successfully on your computer.
- Set up your editor - Install the [Flutter and Dart plugins](#).
- If you have got Android SDK installed and configured, to install Flutter you only need to:
 - Download Flutter SDK from official website and extract it.
 - Add path to previously extracted SDK to your PATH variable
 - Run flutter doctor tool to check if everything is configured correctly.
 - All above steps are mentioned here: <https://flutter.dev/docs/get-started/install/>

Android Studio – Windows

- Download Android Studio - <https://developer.android.com/studio/>
- Get the Flutter SDK - <https://flutter.dev/docs/get-started/install>
- Learn more about Android Studio - <https://developer.android.com/studio/intro/>

Step 1 : Get the Flutter SDK

1 Download the following installation bundle to get the latest stable release of the Flutter SDK:

2 Extract the zip file and place the contained flutter in the desired installation location for the Flutter SDK (for example, C:\src\flutter; do not install Flutter in a directory like C:\Program Files\ that requires elevated privileges).

Step 2 : Update your path

If you wish to run Flutter commands in the regular Windows console, take these steps to add Flutter to the PATH environment variable: From the Start search bar, enter 'env' and select **Edit environment variables for your account**. Under **User variables** check if there is an entry called **Path**:

- If the entry exists, append the full path to **flutter\bin** using ; as a separator from existing values.
- If the entry doesn't exist, create a new user variable named Path with the full path to **flutter\bin** as its value.

Important

Note that you have to close and reopen any existing console windows for these changes to take effect. **You are now ready to run Flutter commands in the Flutter Console!**

Step 3 : Run flutter doctor

From a console window that has the Flutter directory in the path (see above), run the following command to see if there are any platform dependencies you need to complete the setup:

```
c:\src\flutter>flutter doctor
```

Android Studio – macOS

- Download Android Studio - <https://developer.android.com/studio/>
- Download Xcode - <https://apps.apple.com/us/app/xcode/id497799835?mt=12>
- Get the Flutter SDK - <https://flutter.dev/docs/get-started/install>
- Learn more about Android Studio - <https://developer.android.com/studio/intro/>

Step 1 : Get the Flutter SDK

- Download the following installation bundle to get the latest stable release of the Flutter SDK:
- Download SDK and extract downloaded file, just double click on that. and just copy extracted folder and paste it to your desired location (for example, Documents\flutter).

Step 2 : Update your path

Important

Path variable needs to be updated to access “flutter” command from terminal. you can just update path variable for current terminal window only. and if you want to access flutter commands from anywhere in terminal, we need to update SDK path permanently.

To update PATH variable, we need to open terminal.

To update PATH variable for current terminal window only, then enter this command "**export PATH=\$PATH:`pwd`/flutter/bin**" and hit enter key.

To update PATH variable permanently, then Open or create **.bash_profile** file. to open or create that file, then enter "**sudo open -e \$HOME/.bash_profile**" and hit enter key. Append below line to bash_profile file at bottom of all other content. "**export PATH=\$PATH:[PATH_TO_FLUTTER_GIT_DIRECTORY]/flutter/bin**" as [PATH_TO_FLUTTER_GIT_DIRECTORY] is actual path of SDK folder.

Run this command on terminal "**source \$HOME/.bash_profile**" to refresh PATH variables.

Then check whether our SDK is successfully installed or not.

You are now ready to run Flutter commands in the Flutter Console!

Run "**flutter doctor**" into terminal, If you are getting check list of flutter sdk requirements, it means SDK is successfully installed on your machine. and you can start building flutter apps on your machine.

Android Studio – Linux

- Download Android Studio - <https://developer.android.com/studio>
- Get the Flutter SDK - <https://flutter.dev/docs/get-started/install/linux>
- Learn more about Android Studio - <https://developer.android.com/studio/intro/>

Step 1 : Get the Flutter SDK

- Download the following installation bundle to get the latest stable release of the Flutter SDK:
- Download SDK and extract downloaded file, just double click on that. and just copy extracted folder and paste it to your desired location (for example, Documents\flutter).

Step 2 : Update your path

Important

Path variable needs to be updated to access “flutter” command from terminal. you can just update path variable for current terminal window only. and if you want to access flutter commands from anywhere in terminal, we need to update SDK path permanently.

You'll probably want to update this variable permanently, so you can run flutter commands in any terminal session. To update PATH variable, we need to open terminal.

1. Run `source $HOME/.` to refresh the current window, or open a new terminal window to automatically source the file.
2. Verify that the `flutter/bin` directory is now in your PATH by running:

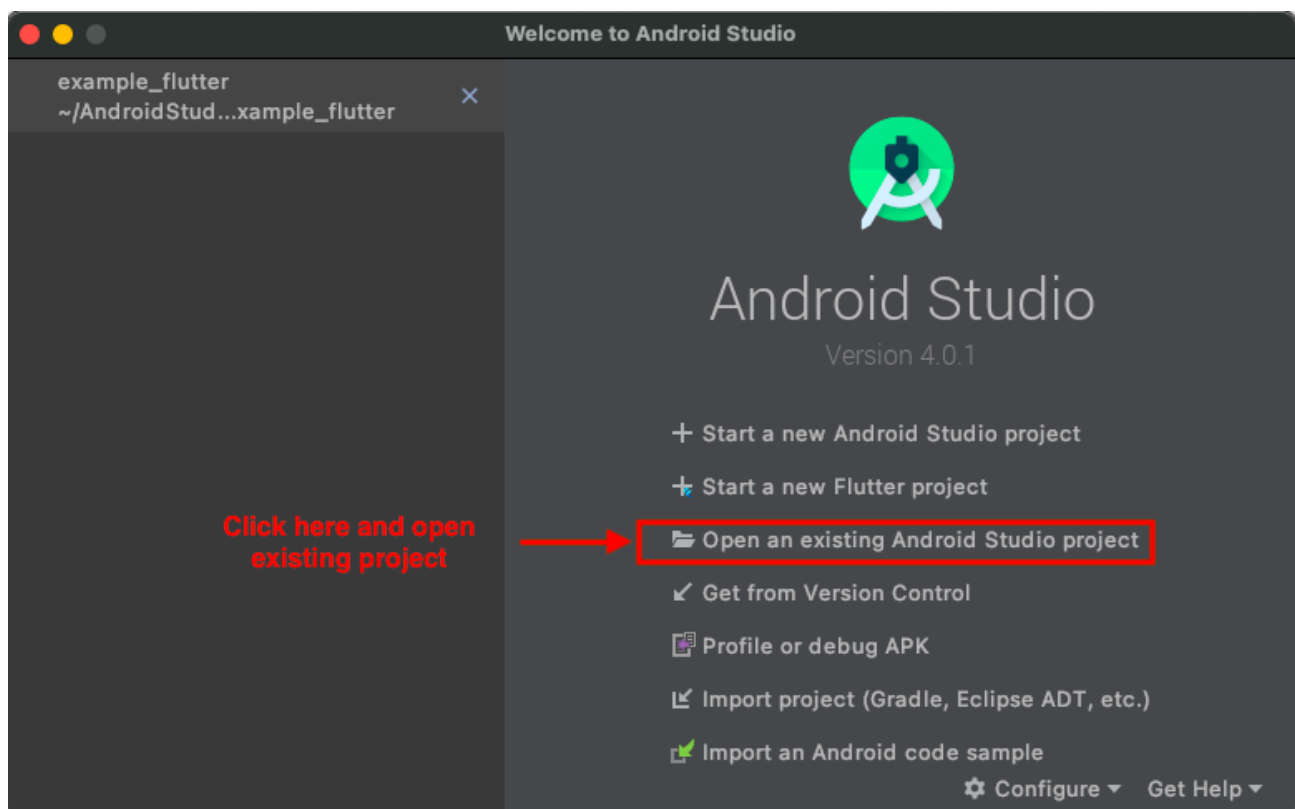
GETTING STARTED (BUILD & RUN)

Important

All below steps are must be followed to build and run application

Download Project

Download and find the your project folder, use your preferred IDE (**Android Studio / Visual Studio Code / IntelliJ IDEA**) to run the project.



logo

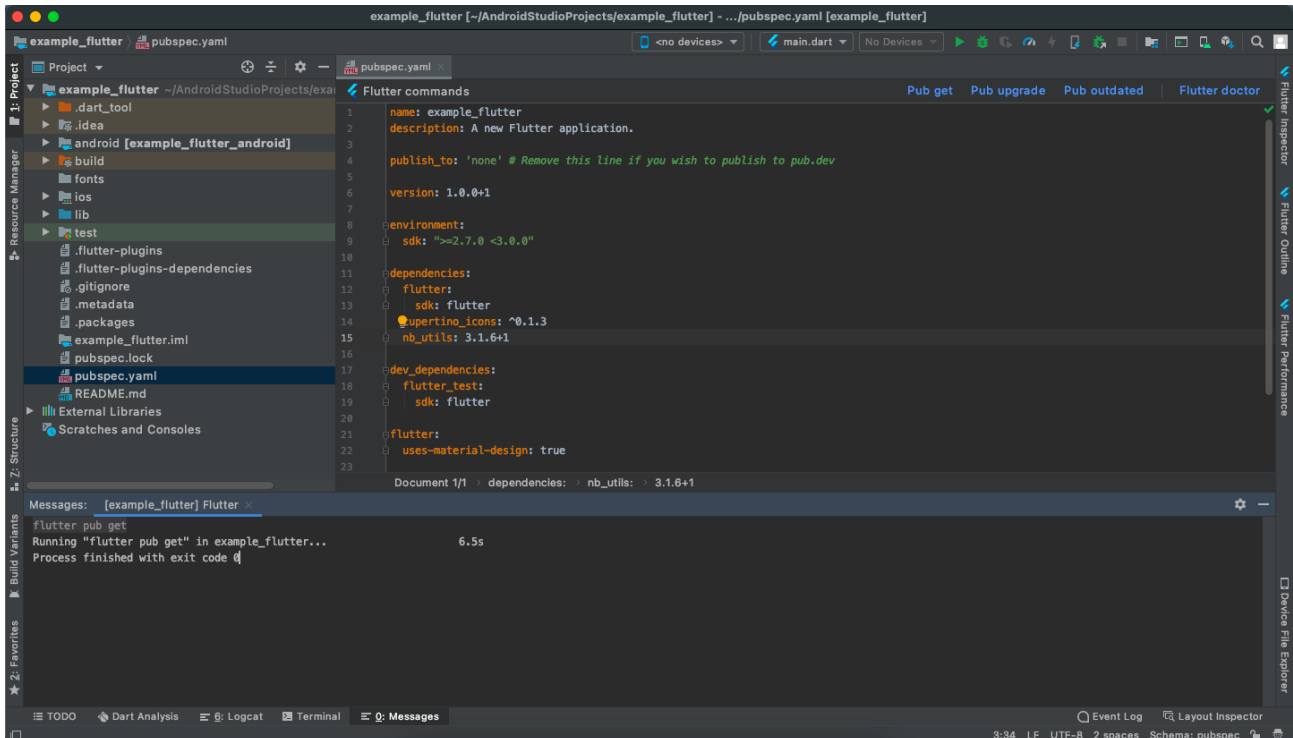
Get Dependencies

After you loaded project successfully, run the following command in the terminal to install all the dependencies listed in the `pubspec.yaml` file in the project's root directory or just click on Pub get in pubspec.yaml file if you don't want to use command.


```
flutter pub get
```

Important

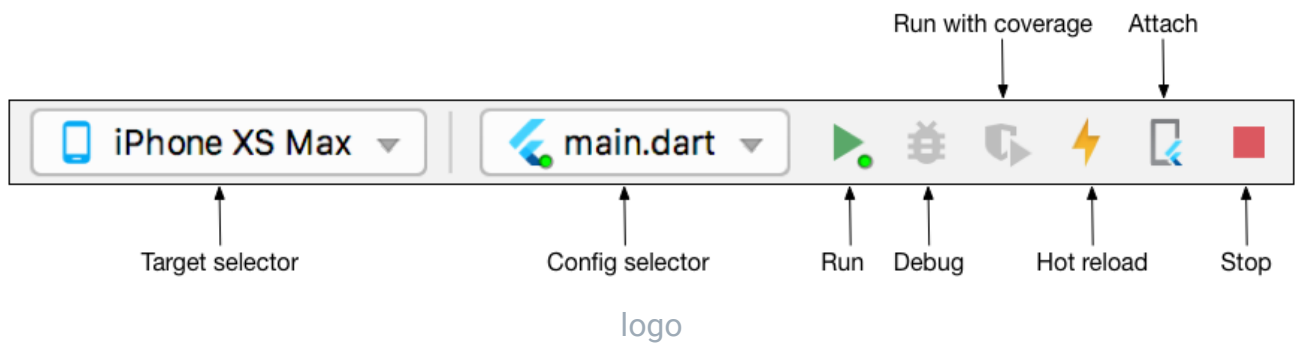
All below steps are must be followed to build and run application



logo

Build and Run App

1. Locate the main Android Studio toolbar.
2. In the target selector, select an Android device for running the app. If none are listed as available, select Tools > Android > AVD Manager and create one there. For details, see [Managing AVDs](#).
3. Click the run icon in the toolbar, or invoke the menu item Run > Run.



After the app build completes, you'll see the app on your device.

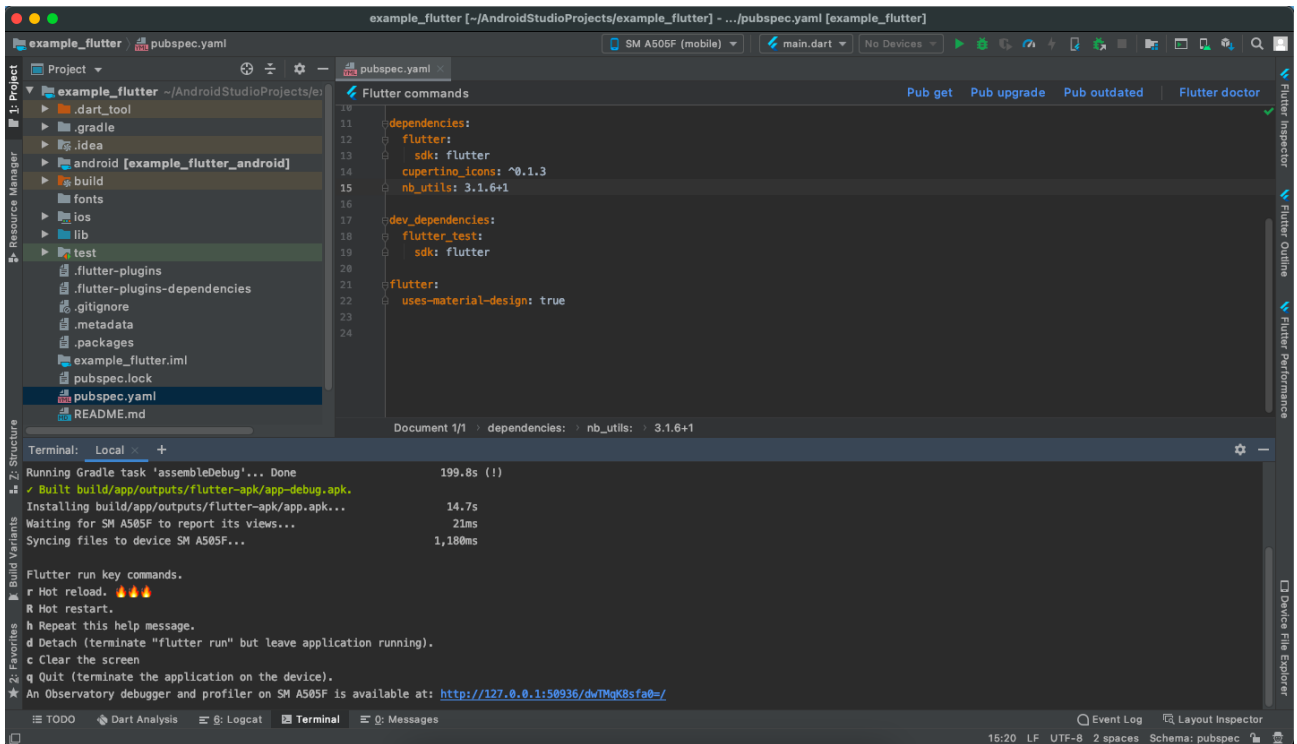
If you don't use Android Studio or IntelliJ you can use the command line to run your application using the following command

Important

Below step requires flutter path to be set in your Environment variables. See <https://flutter.dev/docs/get-started/install/windows>

```
flutter run
```

You will see below like screen after you have build your app successfully.



logo

Try hot reload

Flutter offers a fast development cycle with Stateful Hot Reload, the ability to reload the code of a live running app without restarting or losing app state. Make a change to app source, tell your IDE or command-line tool that you want to hot reload, and see the change in your simulator, emulator, or device.

Important

Do not stop your app. let your app run.

PROJECT STRUCTURE & FEATURES

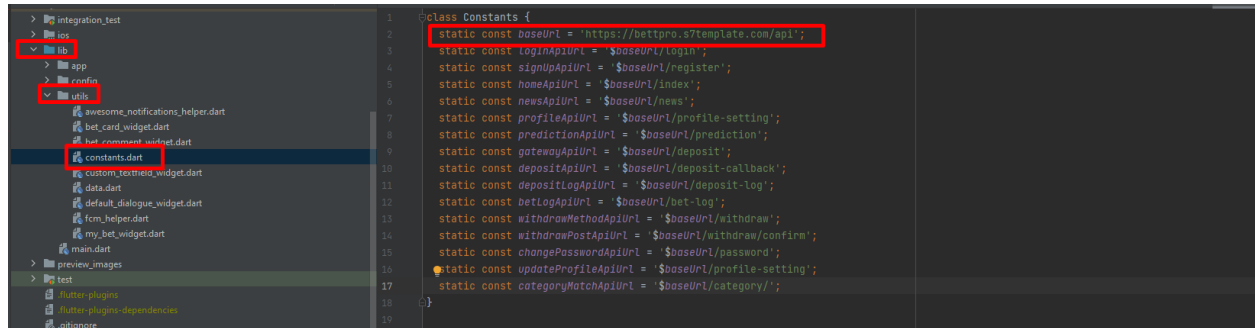
- **images:** apps/widgets/snippets images as well as applcon are stored here.
- **lib:** Application main file and folder are located here.
 - **Screens:** Full Apps UI Code can be found here.
 - **Components:** Re-useable widget can be found here
 - **constant:** All constant value are located here
 - **main:** All the routes are been declared here.
- **pubspec.yaml:** Application name and other project dependencies can be found here.

Application Features

- Clean Code and a well structured project
- Single code base for both Android & iOS
- 60 FPS Support for both Android & iOS
- Fully responsive UI
- Best UI & UX
- Great animation
- Easy to customize
- Free life time updates & stunning customer support
- Easy to integrate in your project

API Setup:

Please go to the file lib > utils> constants.dart and add your website link at the base url variable.



```
1 class Constants {
2   static const baseUrl = 'https://bettpro.s7template.com/api';
3   static const loginApiUrl = '$baseUrl/login';
4   static const signUpApiUrl = '$baseUrl/register';
5   static const homeApiUrl = '$baseUrl/index';
6   static const newsApiUrl = '$baseUrl/news';
7   static const profileApiUrl = '$baseUrl/profile-setting';
8   static const predictionApiUrl = '$baseUrl/prediction';
9   static const gatewayApiUrl = '$baseUrl/deposit';
10  static const depositApiUrl = '$baseUrl/deposit-callback';
11  static const depositLogApiUrl = '$baseUrl/deposit-log';
12  static const betLogApiUrl = '$baseUrl/bet-log';
13  static const withdrawMethodApiUrl = '$baseUrl/withdraw';
14  static const withdrawMethodApiUrl = '$baseUrl/withdraw/confirm';
15  static const changePasswordApiUrl = '$baseUrl/password';
16  static const updateProfileApiUrl = '$baseUrl/profile-setting';
17  static const categoryMatchApiUrl = '$baseUrl/category/';
18 }
```